DEPARTMENT OF SOFTWARE ENGINEERING

UNIVERSITY OF SIALKOT

SOFTWARE DESIGN AND ARCHITECTURE

BSSE-4-Yellow-FALL 2024

SEMESTER PROJECT INSTAGRAM

Group Members:

Qoshina Shahid 24010101_145

Anoosha Aman 24010101_113

Farwa Sagheer24010101_103

Submitted to:

- **Ma'am Irfa**

Table of Contents

Table of Figures:

INSTAGRAM

1. . Introduction

An Instagram Social Media Application is a platform that allows users to connect, communicate, and share their experiences through photos, videos, stories, reels, and messages. The system enables users to create profiles, upload content, interact with other users through likes, comments, and follows, and explore content shared by the community. The application aims to provide an engaging and secure environment for social interaction and digital content sharing while ensuring high performance, reliability, and scalability.

2. Vision Document

2.1 Problem Statement

Category	Description
Problem	• Existing social media applications often struggle with issues such as privacy concerns, fake accounts, content management, and information overload. • Users require a platform that provides secure communication, easy content sharing, and personalized content recommendations. • Managing large numbers of users, posts, and interactions efficiently is challenging. • Poor user experience, slow performance, and weak security mechanisms can reduce user engagement and satisfaction.
Affects	• Users may experience delays in uploading content or accessing feeds. • Privacy breaches and unauthorized access can compromise user trust. • Fake accounts and spam content negatively affect user experience. • Lack of effective communication features limits social interaction.
Impact	• Reduced user engagement and satisfaction. • Loss of user trust due to security and privacy issues. • Increased operational challenges in handling large-scale user activities. • Poor platform performance can damage application reputation.

2.2Solution

- Platform for sharing photos, videos, stories, and reels.
- Account creation and profile management.
- Secure authentication and authorization.
- Like, comment, share, and save functionality.
- Direct messaging system.
- Notifications for activities.
- Privacy and security controls.

2.3Scope

The system will provide:

- User registration and login
- Profile creation and management
- Uploading and sharing photos and videos
- Stories and reels
- Like, comment, share, save features
- Follow/unfollow functionality

- Direct messaging
- Search and explore
- Notifications
- Privacy and security controls

2.4 Constraints

- Internet connectivity dependency
- Server storage limitations
- High traffic handling challenges
- Privacy regulation compliance
- Content moderation requirements
- Security implementation
- Cross-platform compatibility

3. System Features

The Instagram Social Media Application offers:

- User registration and login
- Profile creation and editing
- Posting images and captions
- Liking and commenting
- Following and unfollowing
- Viewing feed
- One-to-one messaging

3.1 Included Artifacts

1. Domain Model
2. Architecture / Design Pattern
3. System Sequence Diagram
4. Sequence Diagram
5. Collaboration Diagram
6. Operation Contracts
7. Design Class Diagram
8. Object Diagram
9. Deployment Diagram
10. Data Flow Diagram
11. Activity Diagram
12. State Transition Diagram
13. Database Model

4. Domain Model

The domain model identifies the major entities and relationships within the Instagram system.

4.1 Main Domain Entities

- User
- Notification
- Profile
- Post
- Comment
- Like

- Message

4.2 Relationships

- A User owns one Profile.
- A User creates multiple Posts.
- A User can Like multiple Posts.
- A User can Comment on multiple Posts.
- A User can Send and Receive Messages.

4.3 Domain Model Purpose

The domain model provides a high-level understanding of how data and business entities interact within the Instagram platform. It serves as the foundation for further design activities.

4.4 Domain Model Diagram Notations

- **Entity/Class** → Rectangle representing a domain object.
- **Association** → Line connecting related entities.
- **Multiplicity** → Indicates how many instances participate in a relationship.
- **Attribute** → Property belonging to an entity.

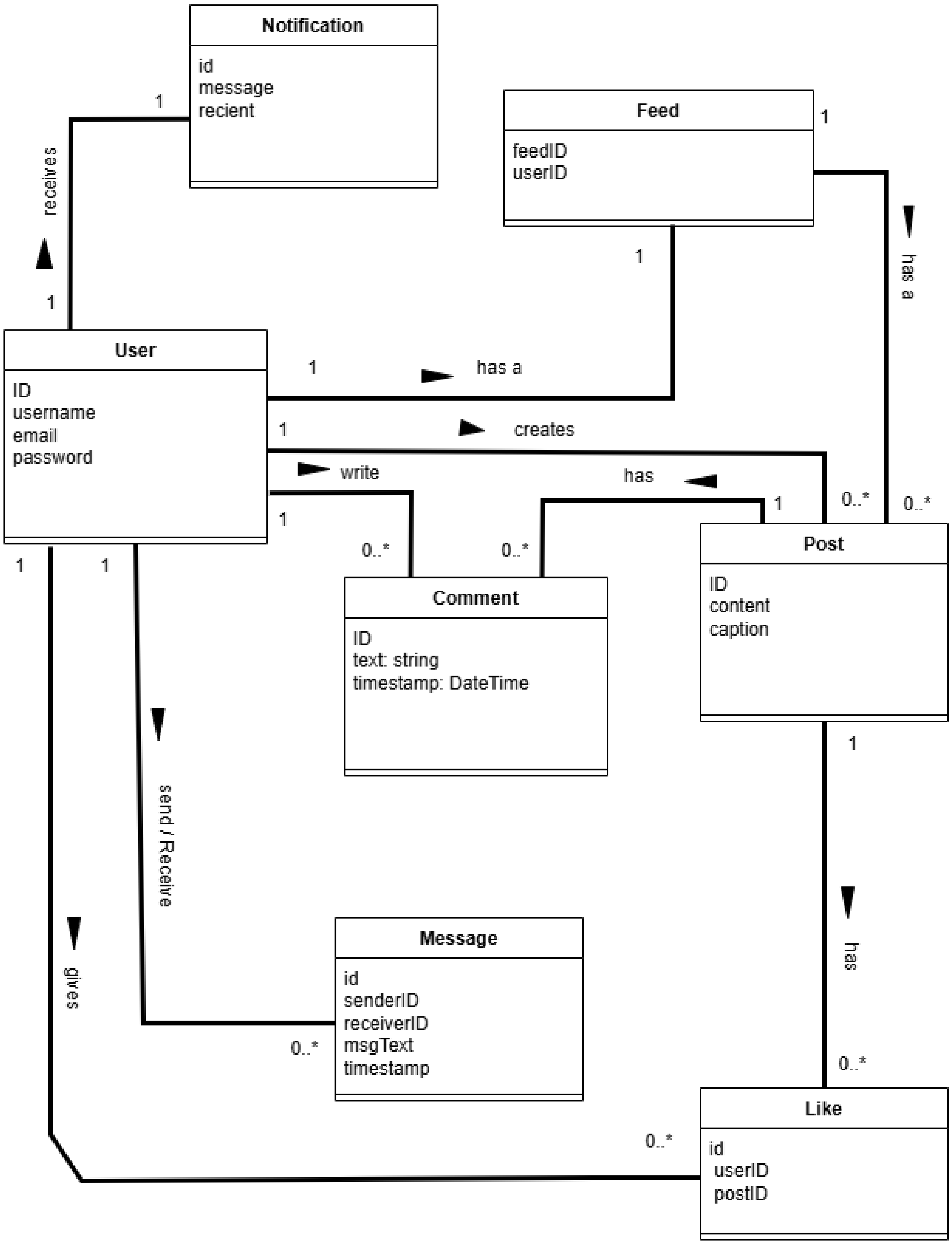


Figure 1: Domain model

5. Architecture / Design Pattern

5.1 Architectural Style:

The Instagram system follows a Layered Architecture Pattern.

5.1.1 Layered Architecture:

Presentation Layer

Responsible for:

- User Interface
- Login Screens
- Feed Display
- Profile Management
- Messaging Interface

Business Logic Layer

Responsible for:

- Authentication
- Feed Generation
- Post Management
- Follow/Unfollow Processing
- Messaging Logic

Data Access Layer

Responsible for:

- Data Retrieval
- Data Storage
- Database Communication

Database Layer

Stores:

- User Data
- Posts
- Comments
- Likes
- Followers
- Messages

Justification

The layered architecture:

- Improves maintainability
- Supports scalability
- Enhances code reusability
- Simplifies debugging
- Allows future feature expansion

5.2 MVC Design Pattern

The Instagram system can also utilize the MVC Pattern.

Model Contains:

- User
- Post
- Comment
- Message
- Feed

View

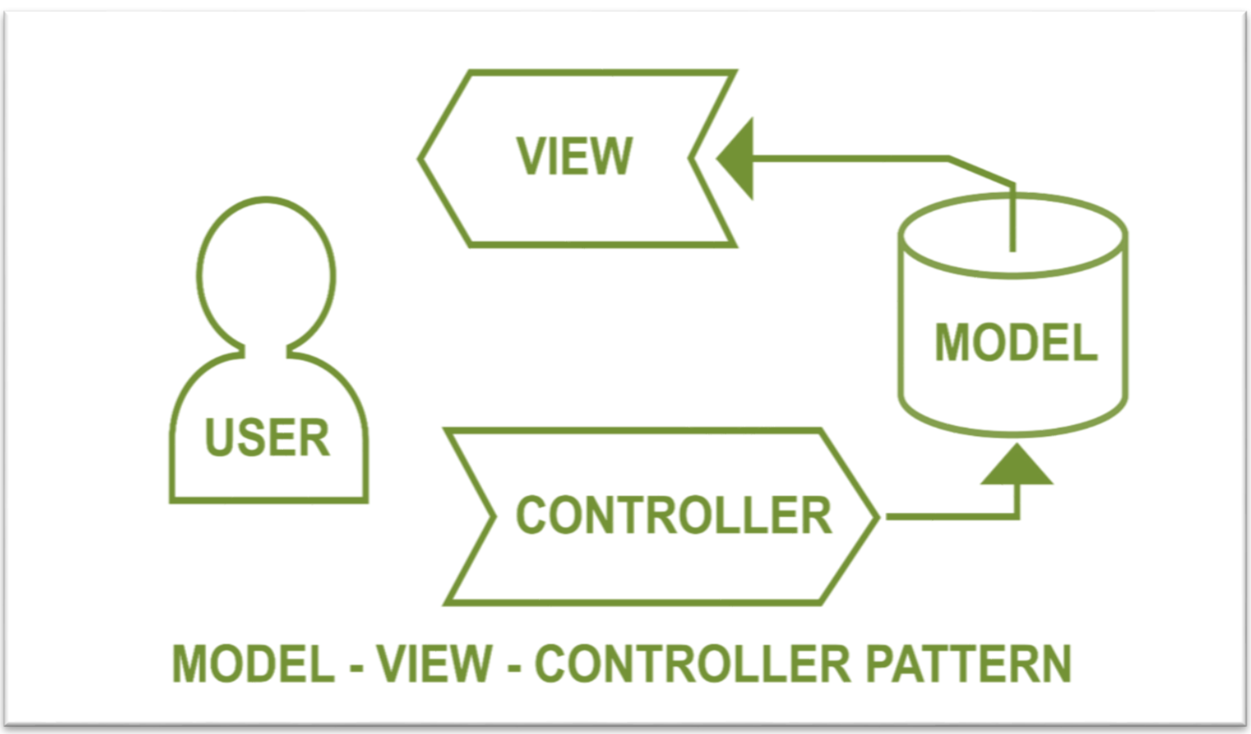
Provides:

- User Interface
- Responsive Screens
- Profile Pages
- Feed Pages

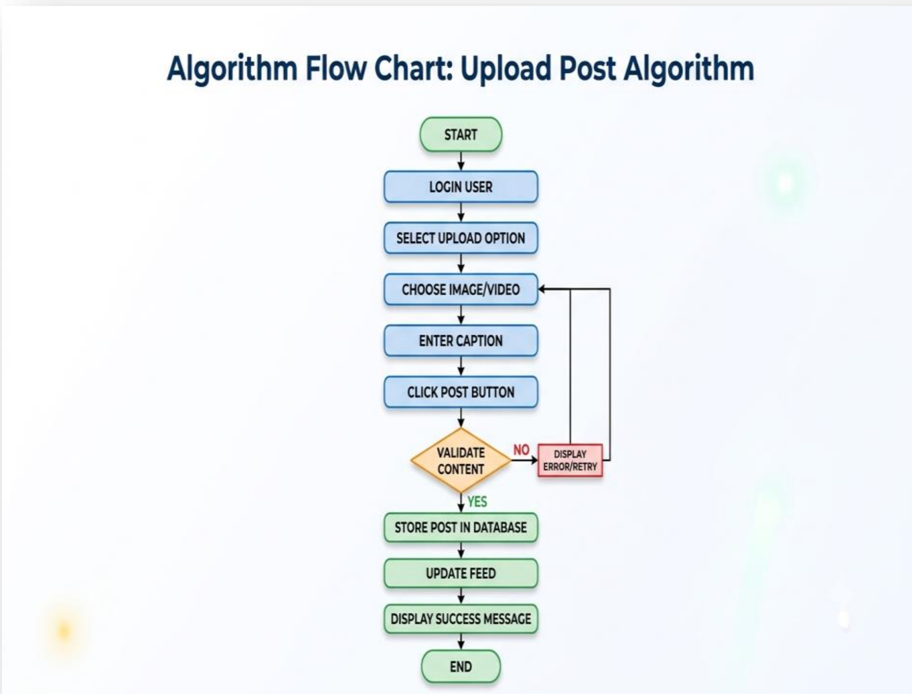
Controller

Handles:

- User Requests
- Business Logic Calls
- Navigation Between Views



5.3 Algorithm Flow Chart



6. Use Case Diagram

A Use Case Diagram is a UML (Unified Modeling Language) diagram used to represent the functional requirements of a system from the user's perspective. It shows who interacts with the system (actors) and what services or functions the system provides (use cases).

6.1 Main Elements (Notations) of a Use Case Diagram

- **Actor:** An actor represents a user, administrator, or external system that interacts with the Instagram Social Media Application.
- **Use Case:** A use case represents a function or service provided by the system such as login, upload post, follow users, or send messages.
- **System Boundary:** A rectangle surrounding all use cases that defines what belongs to the Instagram system.
- **Association Relationship:** Shows communication between actors and use cases.
- **Include Relationship:** Used when one use case always requires another use case.
- **Extend Relationship:** Used when additional behavior occurs under specific conditions.
- **Generalization (Inheritance):** Used when actors or use cases share common behavior.

6.2 Use Cases:

The major use cases include:

- Register Account
- Login
- Manage Profile
- Upload Photos and Videos
- Create Stories and Reels
- Like Posts
- Comment on Posts
- Follow Users
- Send Direct Messages
- Search Users and Posts
- Receive Notifications
- Report Content
- Manage Privacy Settings
- Logout

6.3 Actor

User

6.4 System Events

1. User selects upload option.
2. User chooses image/video.
3. User enters caption.
4. User clicks post button.

5. System validates data.
6. System stores post.

- System updates feed.
- System displays confirmation.

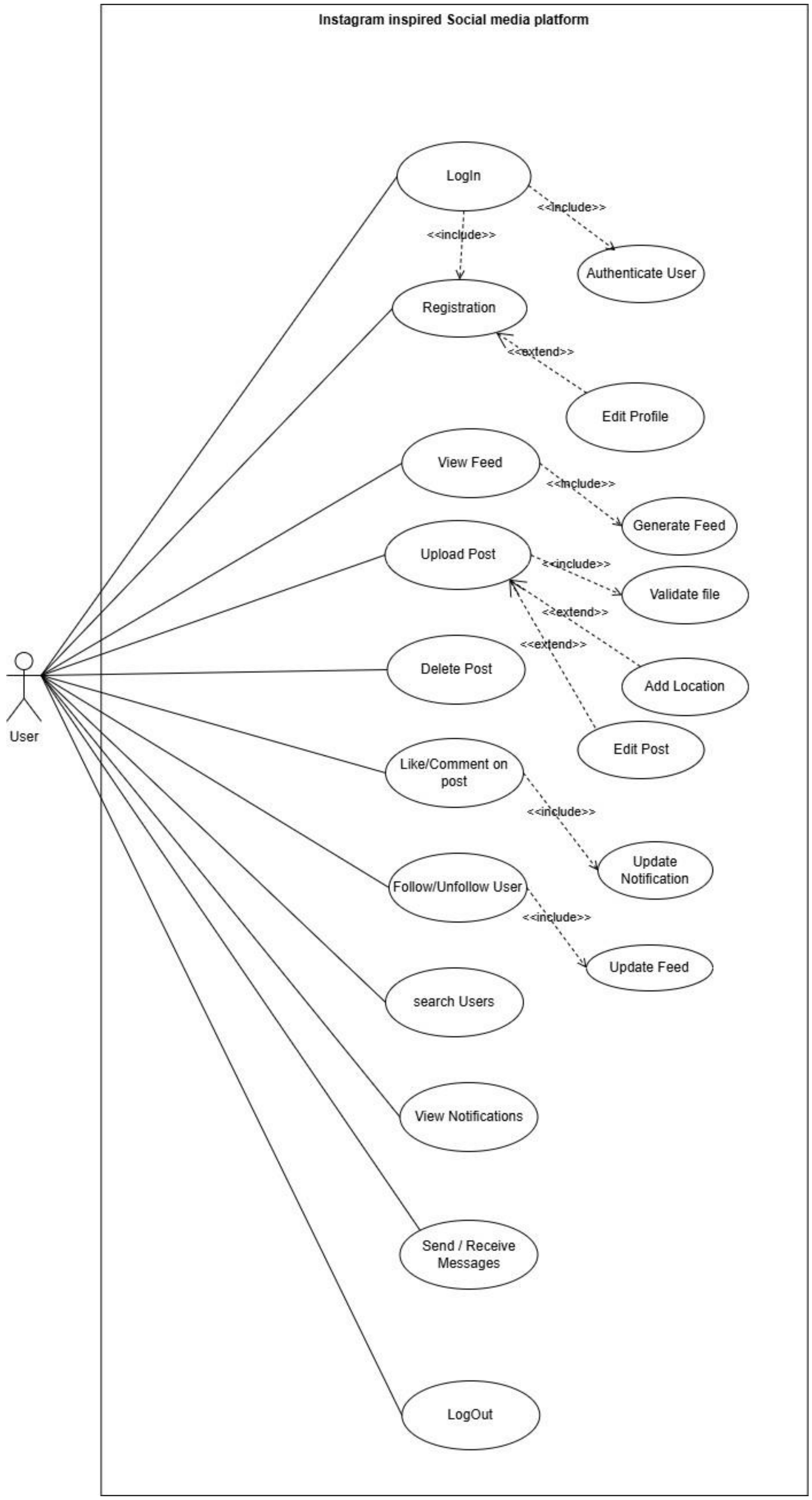


Figure 2: Use Case Diagram

Use Case description for post upload:

Use Case ID	**UC-03**
Use Case Name	**Upload Post**
Actor	**User**
Description	**This use case describes how a user uploads an image with a caption to share on their profile. The content will be saved to the system and will be visible in the feed of their followers.**
Pre-condition	**The user must be registered and logged in.**
Post-condition	**The post is stored in the system and is visible to the user and their followers.**
Normal Flow	• **User navigates to the upload section.** • **User selects an image or video file.** • **User adds a caption (optional).** • **User taps the "Post" button.** • **The system stores the content in the Post Database.** • **The system confirms post success and refreshes the feed.**
Exit Condition	**The user returns to the home/feed page after successfully uploading the post.**
Exception	• **File not supported → Error message shown.** • **Upload fails due to network issue → Retry.**

7. Sequence Diagram

7.1 Objects

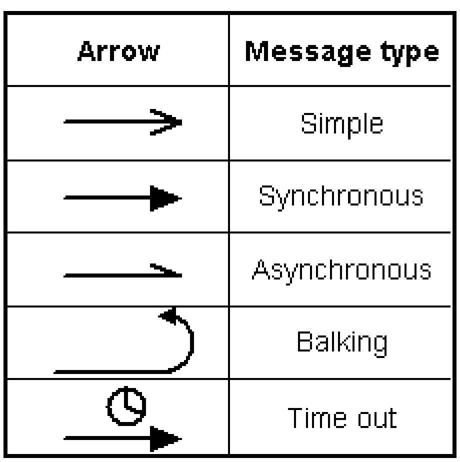
- User
- Upload Interface
- Post Controller
- Post Service
- Database

7.2 Sequence

1. User → Upload Interface: Select Media
2. Upload Interface → Post Controller: Submit Post
3. Post Controller → Post Service: Validate Data
4. Post Service → Database: Save Post
5. Database → Post Service: Success
6. Post Service → Post Controller: Confirmation
7. Post Controller → Upload Interface: Display Success

7.3 Messages

Messages are arrows that represent communication between objects. Use half-arrowed lines to represent asynchronous messages. Asynchronous messages are sent from an object that will not wait for a response from the receiver before continuing its tasks.



Arrow	Message type
	Simple
	Synchronous
	Asynchronous
	Balking
	Time out

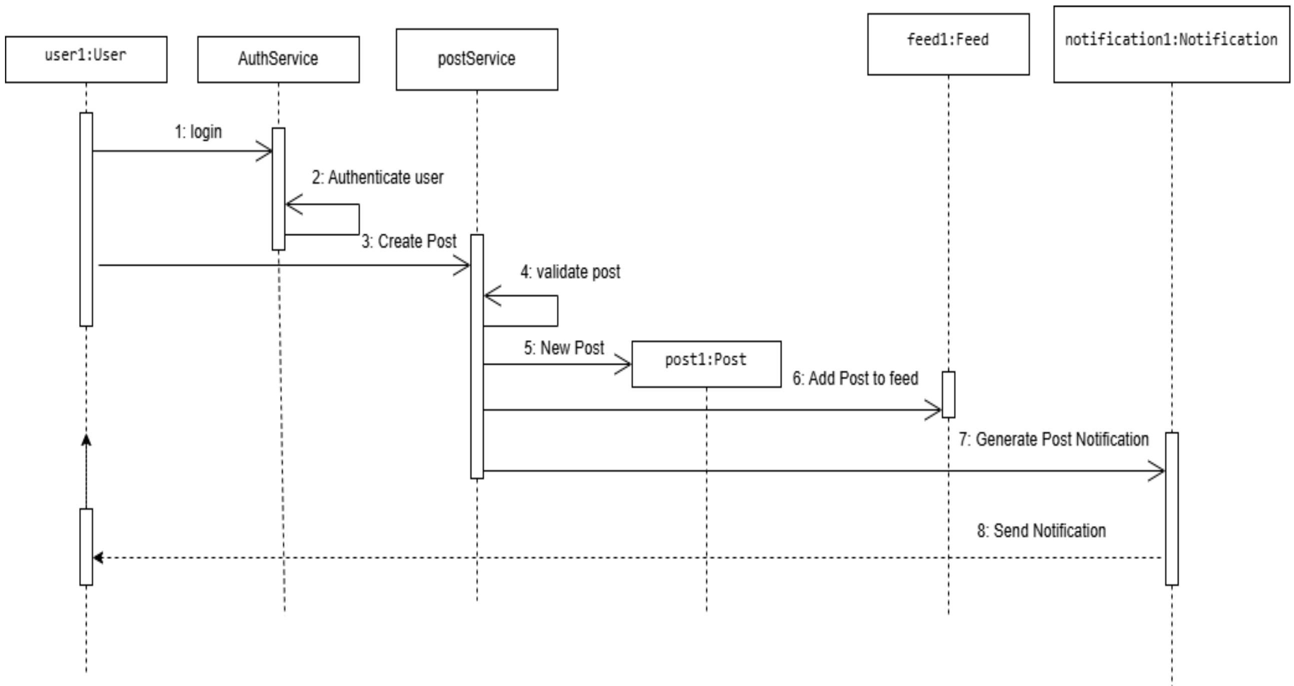


Figure 3: Sequence Diagram

8. Collaboration Diagram

A Collaboration Diagram shows how different parts of a system work together and communicate to perform a process.

For our **Instagram Social Media Application**, the collaboration diagram describes how the User, Authentication System, Application Server, Database Server, Media Storage Service, Notification Service, and Messaging Service collaborate to register users, upload posts, interact with content, exchange messages, and manage notifications.

8.1 Participating Objects

- User
- Upload Interface
- Post Controller
- Post Service
- Database

8.2 Message Flow

Select Media

Add Caption

Submit Post

Validate Content

Save Post

Update Feed

Return Success Message

8.3 Main Elements (Notations) of a Collaboration Diagram

- Actor: Represents a user or external entity interacting with the system.
- Object: Represents a participant involved in the collaboration.
- Link: Represents a connection between objects that communicate with each other.
- Message: Represents communication between objects.
- Message Numbering: Represents the sequence in which messages are exchanged.
- Synchronous Message: Represents a message where the sender waits for a response.
- Asynchronous Message: Represents a message where the sender continues execution without waiting for a response.
- Self-Message: Represents communication within the same object.
- Return Message: Represents a response returned to the sender.
- Association Link: Represents a relationship between collaborating objects.
- Navigation Direction: Represents the direction of message flow.
- Object Role: Represents the responsibility of an object within the collaboration.

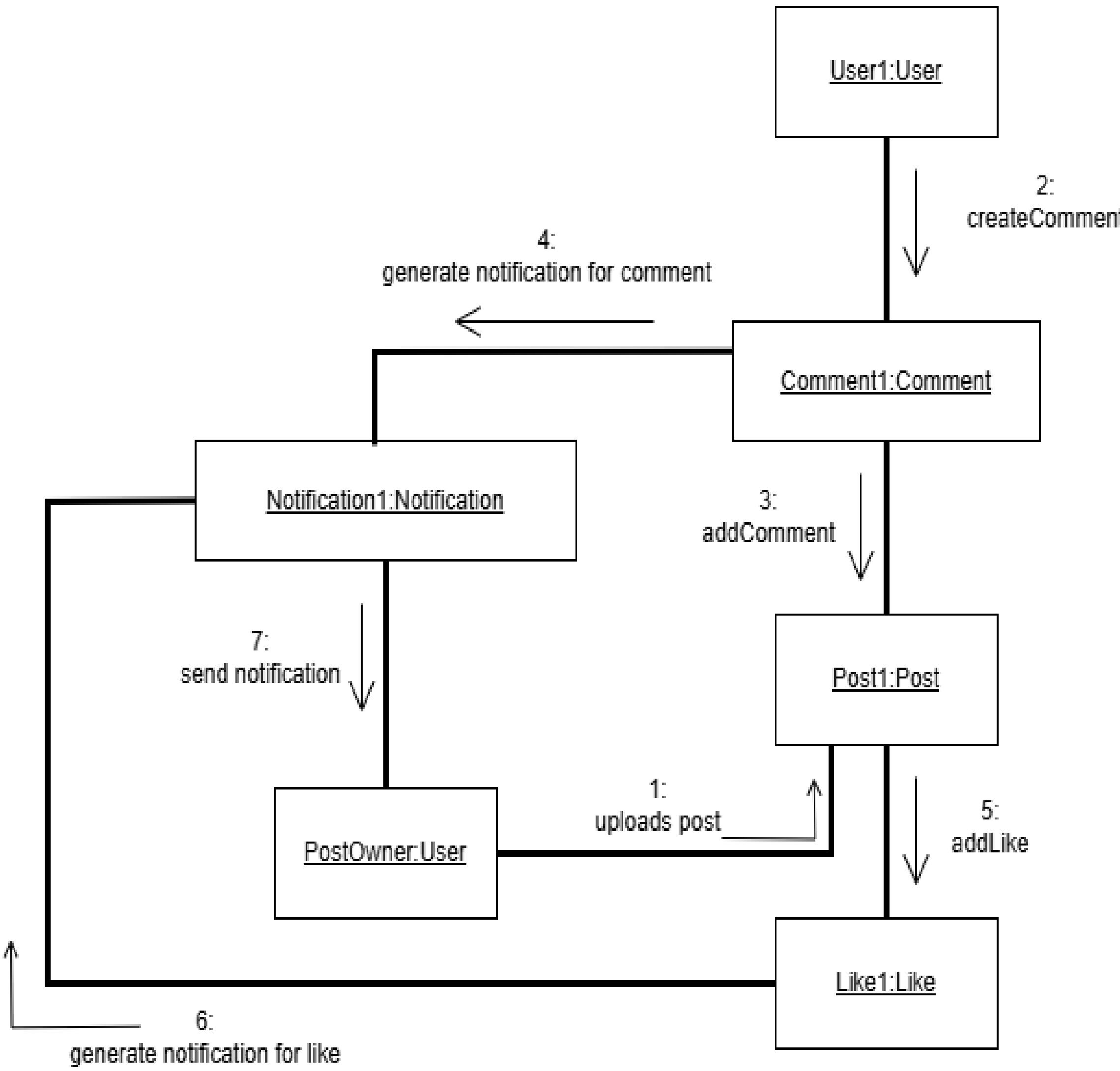


Figure 4: Collaboration Diagram

9. Operation Contracts

9.1 Operation Contract: Upload Post

Name
UploadPost()

Responsibilities
Store image/video content and caption.

Cross References
Use Case: Upload Post

Preconditions
- User is registered.
- User is logged in.

Postconditions
- New Post object created.
- Post stored in database.
- Feed updated.
- Followers can view post.

Exceptions
- Unsupported file format.
- Upload failure.
- Network issue.

9.2 Operation Contract: Follow User

Name
FollowUser()

Responsibilities
Allow a user to follow another user.

Preconditions
- Both users exist.
- User is logged in.

Postconditions

- Follow relationship created.
- Feed updated.

9.3 Operation Contract: Send Message

Name

SendMessage()

Responsibilities

Send text message to another user.

Preconditions

- Sender logged in.
- Receiver exists.

Postconditions

- Message stored.
- Receiver can view message.

10. Class Diagram

A Class Diagram represents the static structure of the system by showing classes, attributes, methods, and relationships.

10.1 Classes

1. User

Attributes

- userID
- username
- email
- password

Methods

- login()
- logout()
- register()

2. Profile

Attributes

- profileID
- bio
- profilePicture

Methods

- editProfile()

3. Post

Attributes

- postID
- caption
- mediaURL
- datePosted

Methods

- uploadPost()
- deletePost()

4. Comment

Attributes

- commentID
- commentText

Methods

- addComment()

5. Like

Attributes

- likeID

Methods

- likePost()

6. Message

Attributes

- messageID
- messageText
- timestamp

Methods

- sendMessage()

7. Feed

Attributes

- feedID

Methods

- generateFeed()

10.2 Class Diagram Notations

- **Class** → A rectangle divided into three sections representing class name, attributes, and methods.
- **Association** → A straight line showing relationship between two classes.
- **Inheritance (Generalization)** → A line with a hollow triangle showing parent-child relationship.
- **Aggregation** → A line with a hollow diamond showing weak ownership.
- **Composition** → A line with a filled diamond showing strong ownership.
- **Multiplicity** → Numbers (1, *, 0..1) showing the number of objects involved in a relationship.

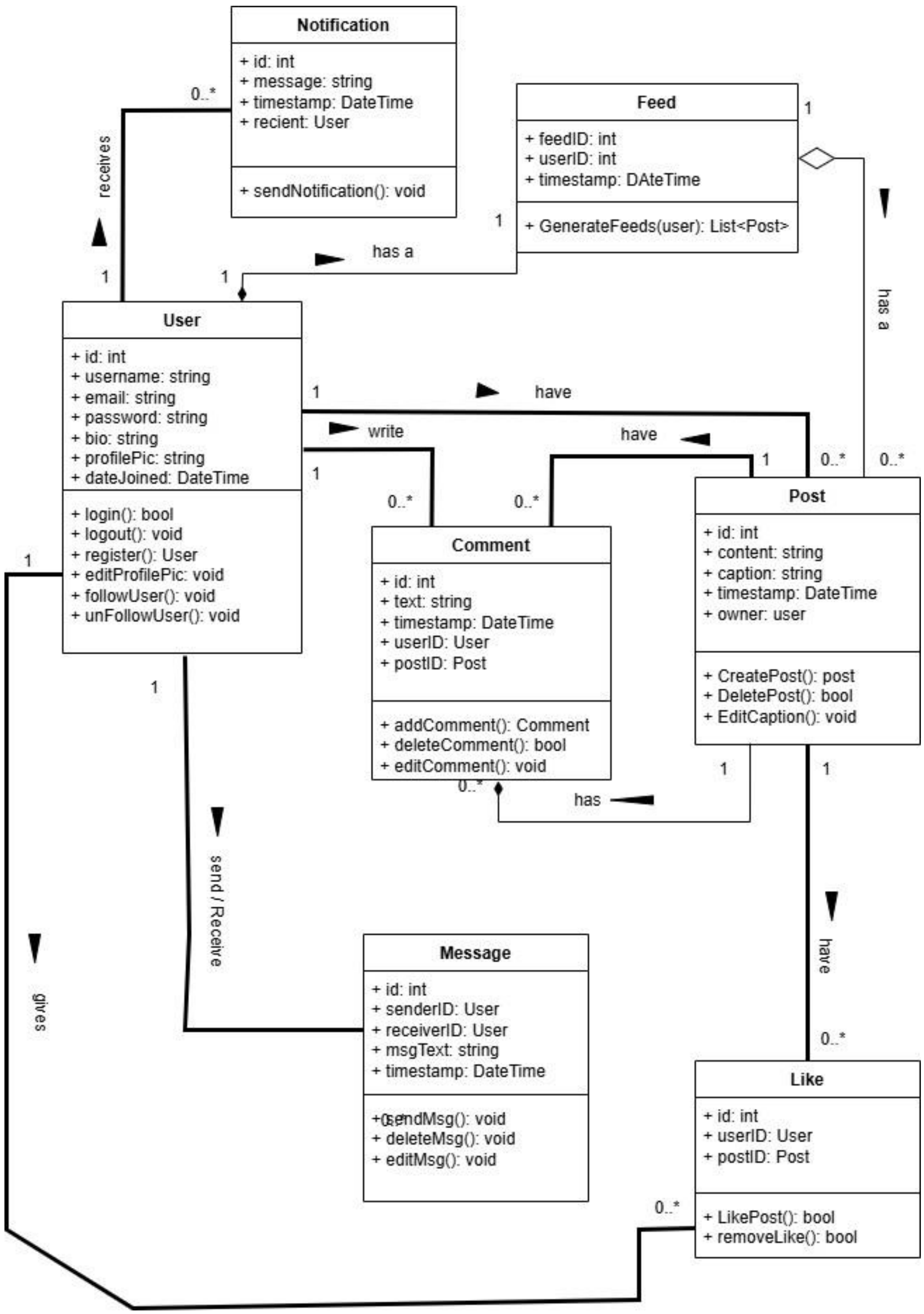


Figure 5: Class Diagram

11. Deployment Diagram

Deployment Diagram shows how software components are deployed on hardware. It helps understand the physical architecture of the system.

11.1 Application in Instagram

Nodes:

- User Mobile Device
- Web Server
- Application Server
- Database Server

Shows where Instagram components are deployed.

11.2 Deployment Diagram Notations

- **Node** → 3D box representing hardware or execution environment.
- **Artifact** → Physical software deployed on a node.
- **Communication Path** → Line showing communication between nodes.
- **Device Node** → Represents hardware devices such as servers or mobiles.

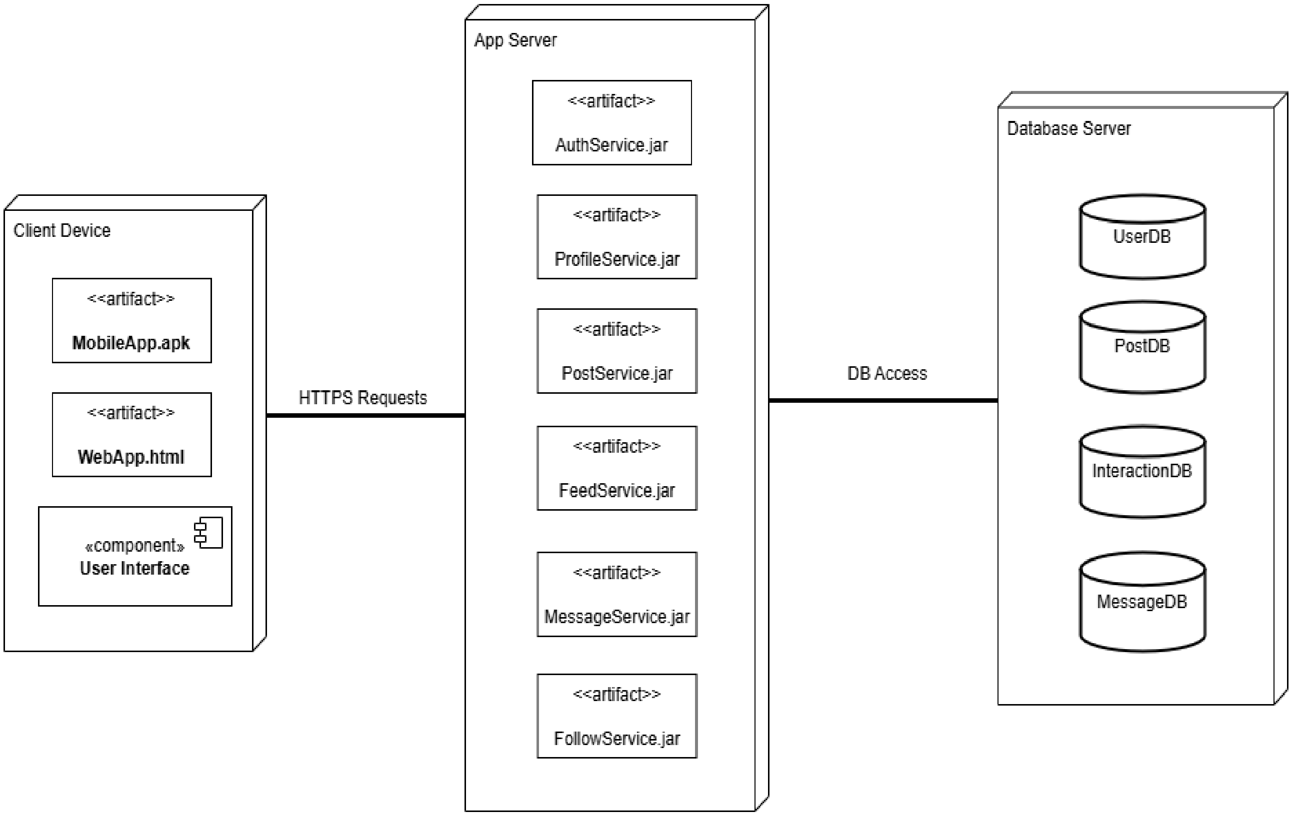


Figure 6: Deploment Diagram

12. Dataflow Diagram (DFD)

A Data Flow Diagram shows how data enters the system, how it is processed, where it is stored, and where it goes.

For our Instagram Social Media Application, the DFD describes how user information, posts, comments, messages, notifications, and media content flow between users and system components.

Level 0

User ↔ Instagram System

Level 1

Processes:

- Login
- Profile Management
- Posting
- Feed Generation
- Messaging

Level 2

Detailed flow of Upload Post process.

12.1 Data Flow Diagram (DFD) Notations

- **External Entity** → Rectangle representing external user/system.
- **Process** → Circle or rounded rectangle representing data processing.
- **Data Store** → Open-ended rectangle representing stored data.
- **Data Flow** → Arrow representing movement of data.

Level 0

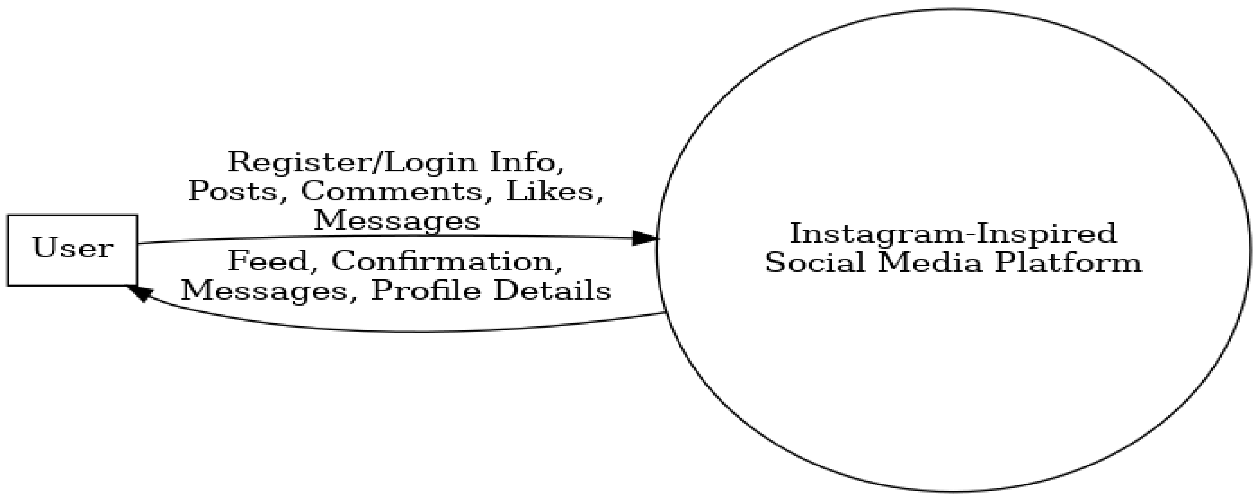


Figure 7: DFD level 0

Level 1

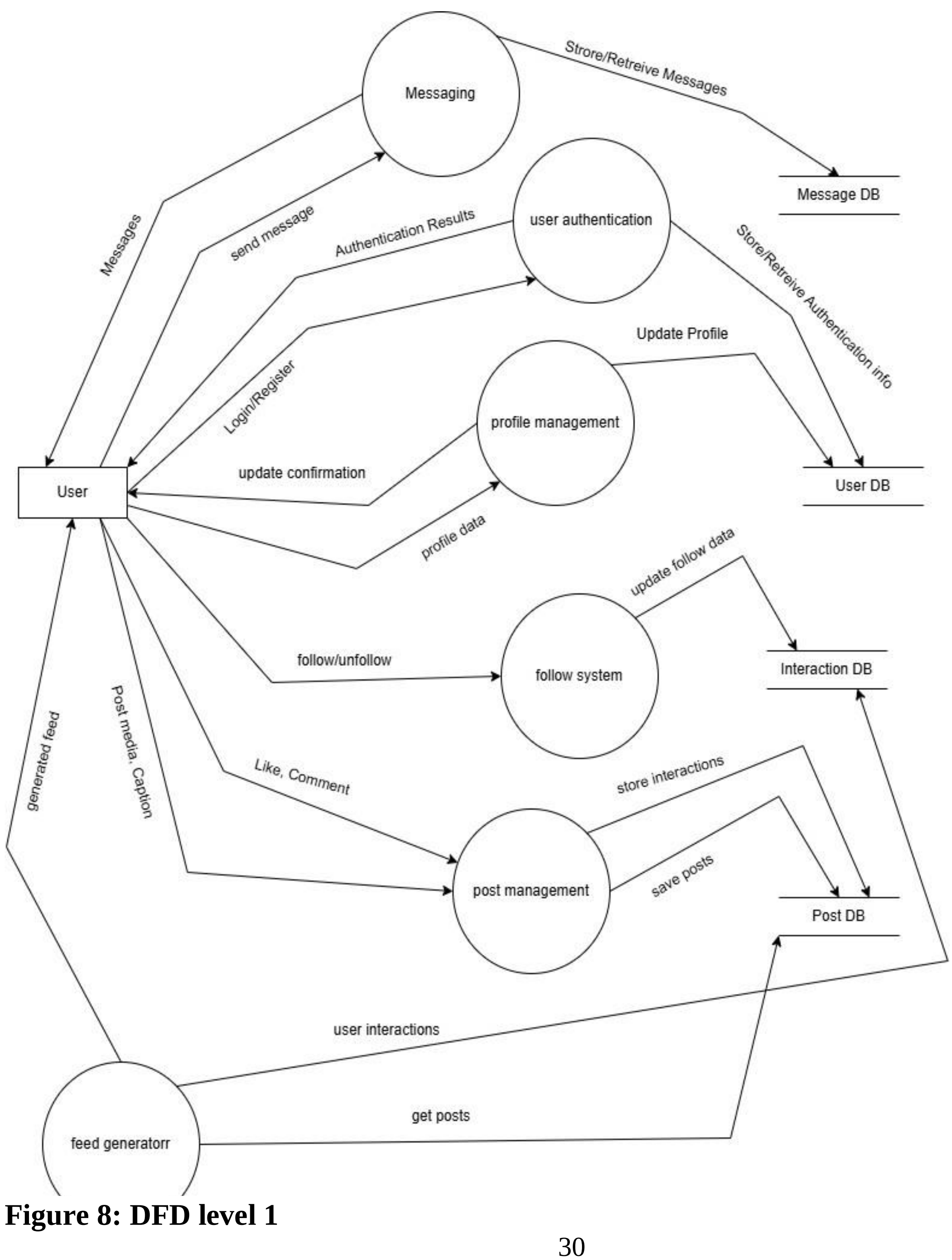


Figure 8: DFD level 1

Level 2

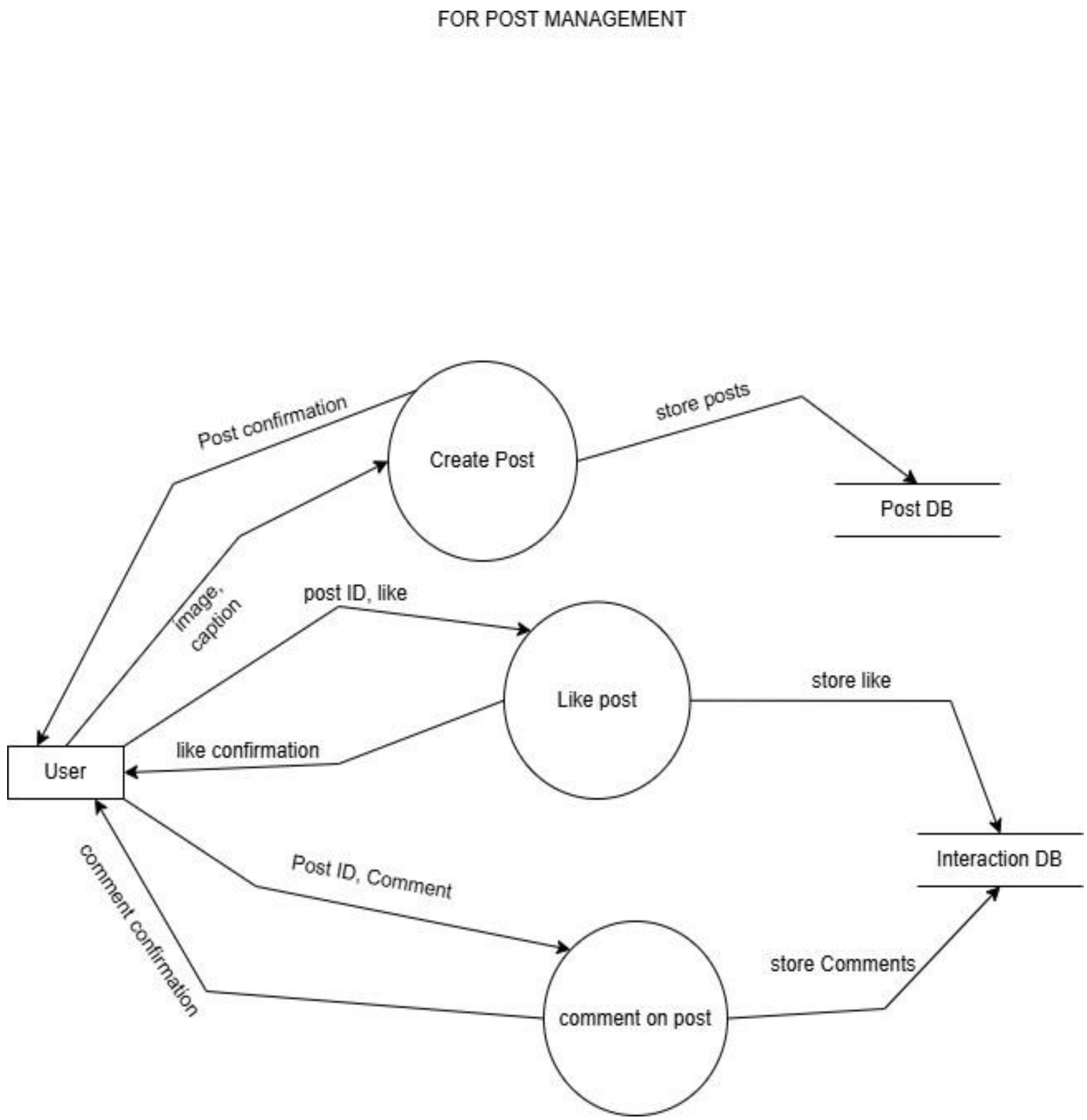


Figure 9: DFD level 2

13. Activity Diagram

Activity Diagram represents workflow and activities. It helps visualize business processes.

For our **Instagram Social Media Application**, the Activity Diagram describes how users register accounts, log in, upload posts, interact with content through likes and comments, follow other users, send messages, and receive notifications.

13.1 Application in Instagram

Upload Post Activity:

Start

↓

Select Media

↓

Add Caption

↓

Validate

↓

Post Content

↓

Update Feed

↓

End

13.2 Activity Diagram Notations

- **Initial State** → Filled black circle.
- **Activity** → Rounded rectangle showing action.
- **Decision Node** → Diamond for branching conditions.
- **Fork Node** → Thick line splitting parallel activities.
- **Join Node** → Thick line combining parallel activities.
- **Final State** → Bull's-eye symbol indicating process completion.

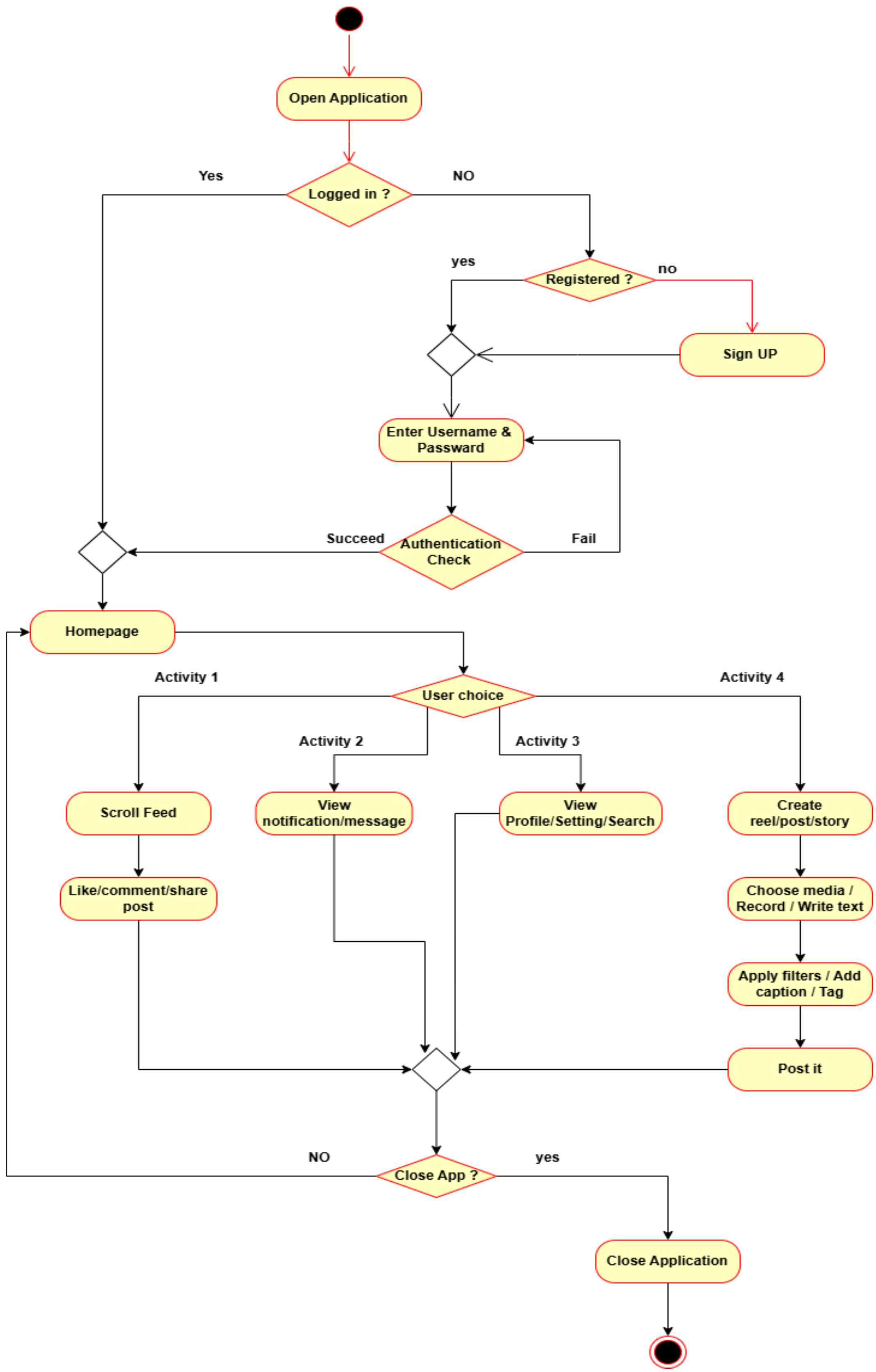


Figure 10: Activity Diagram

14. State Chart Diagram

A State Chart Diagram represents the different states of a system and transitions between them based on events or actions.

For our Instagram Social Media Application, the State Chart Diagram describes how the application transitions between registration, login, posting content, messaging, notifications, and logout states.

14.1 States of our System

- Initial State
- Registration State
- Login State
- Profile Management State
- Feed Browsing State
- Post Creation State
- Story/Reel Upload State
- Like and Comment State
- Messaging State
- Notification State
- Search State
- Logout State
- Final State

14.2 State Diagram Notations

- **Initial State** → Filled black circle.
- **State** → Rounded rectangle representing a condition.
- **Transition** → Arrow showing movement between states.
- **Event** → Trigger causing transition.
- **Final State** → Bull's-eye symbol indicating termination.

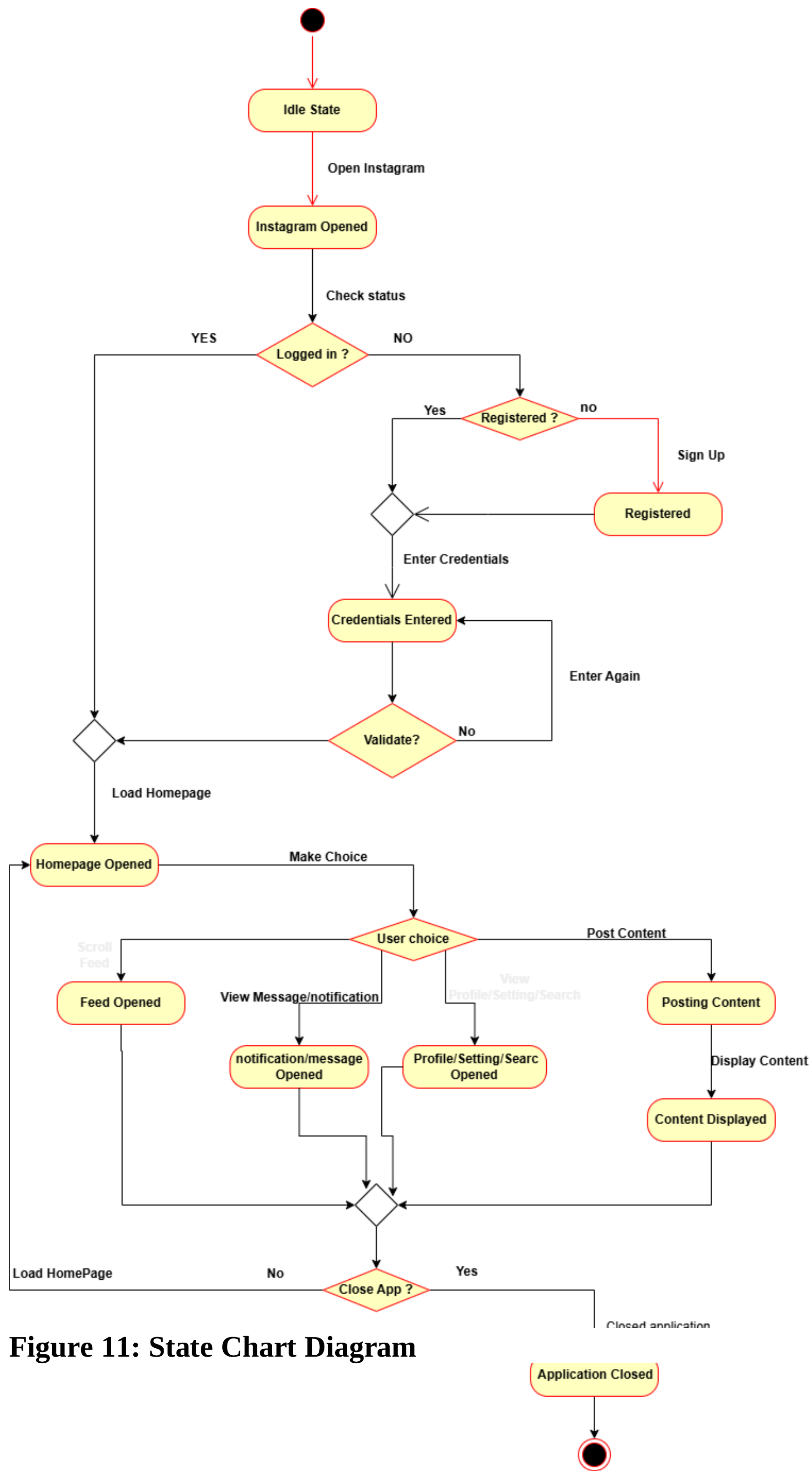


Figure 11: State Chart Diagram

15. Database Model

User Table

Attribute	Type
UserID	INT
Username	VARCHAR
Email	VARCHAR
Password	VARCHAR

Profile Table

Attribute	Type
ProfileID	INT
UserID	INT
Bio	TEXT
ProfilePicture	VARCHAR

Post Table

Attribute	Type
PostID	INT
UserID	INT
Caption	TEXT
MediaURL	VARCHAR
DatePosted	DATETIME

Comment Table

Attribute	Type
CommentID	INT
PostID	INT
UserID	INT
CommentText	TEXT

Like Table

Attribute	Type
LikeID	INT
PostID	INT
UserID	INT

Follow Table

Attribute	Type
FollowID	INT
FollowerID	INT
FollowingID	INT

Message Table

Attribute	Type
MessageID	INT
SenderID	INT
ReceiverID	INT
MessageText	TEXT
Timestamp	DATETIME